

UML — Unified Modeling Language aplicada ao RadarTorres

1 Introdução à UML

A UML (*Unified Modeling Language*, ou Linguagem de Modelagem Unificada) é uma notação gráfica padronizada utilizada para visualizar, especificar, construir e documentar os artefatos de um sistema de software [REFERÊNCIA BIBLIOGRÁFICA A INSERIR]. Sua finalidade não é gerar código nem substituir a implementação, mas oferecer uma representação intermediária, comum a todos os envolvidos no desenvolvimento, capaz de expressar tanto a estrutura estática de um sistema (quais elementos existem e como se relacionam) quanto seu comportamento dinâmico (como esses elementos interagem ao longo do tempo).

Uma característica central da UML é sua independência em relação à linguagem de programação, ao banco de dados e à arquitetura tecnológica adotados. Um mesmo diagrama de classes pode, em princípio, descrever um sistema implementado em C#, Java ou Python; o que a UML fixa é o vocabulário conceitual (classe, atributo, associação, ator, caso de uso, nó de implantação), não a sintaxe de nenhuma linguagem específica. Essa independência é o que permite à UML atuar como ponte entre a especificação de requisitos e o projeto técnico do sistema: um mesmo conjunto de diagramas serve tanto para comunicar decisões a stakeholders não técnicos quanto para orientar a equipe de desenvolvimento durante a implementação.

Os diagramas UML costumam ser agrupados em duas grandes categorias. Os **diagramas estruturais** (dos quais o diagrama de classes e o diagrama de implantação, tratados neste documento, são exemplos) descrevem os elementos que compõem o sistema e as relações estáticas entre eles — o que existe, independentemente do tempo. Os **diagramas comportamentais** (dos quais o diagrama de casos de uso é um exemplo) descrevem como o sistema se comporta e como interage com atores externos — o que acontece, ao longo de uma execução ou de um fluxo de interação [REFERÊNCIA BIBLIOGRÁFICA A INSERIR]. Um projeto de software normalmente combina diagramas dos dois grupos, cada um evidenciando um aspecto diferente do mesmo sistema.

2 UML aplicada ao RadarTorres

O RadarTorres é um sistema com características que tornam a modelagem UML particularmente útil. Trata-se de uma aplicação desktop em C#/.NET 9, com interface gráfica em WPF, que integra duas naturezas de sistema normalmente tratadas de forma isolada:

um software convencional de gestão (autenticação, controle de acesso por perfil, auditoria, persistência de dados, preferências de usuário) e um sistema de tempo real que se comunica com hardware externo (o Arduino, responsável pelos sensores de detecção e pelo acionamento das torres demonstrativas) por meio de comunicação serial.

Essa dupla natureza é exatamente o que os diagramas estruturais e comportamentais da UML permitem representar de forma complementar. O diagrama de classes evidencia a estrutura lógica do software — como as entidades de domínio (alvo, torre, zona morta, usuário, registros de auditoria) e os serviços que as manipulam estão organizados internamente. O diagrama de casos de uso evidencia a interação entre os atores (humanos, com diferentes perfis de acesso, e o próprio Arduino) e o sistema, sem se preocupar com como essa interação é implementada. O diagrama de implantação, por sua vez, evidencia um aspecto que um sistema puramente de software não precisaria representar: a topologia física real, incluindo o computador Windows, o microcontrolador Arduino e a comunicação USB/serial que os conecta.

Nota sobre a arquitetura. O RadarTorres não utiliza o padrão arquitetural MVC (*Model-View-Controller*). Sua arquitetura é **MVVM** (*Model-View-ViewModel*), padrão natural para aplicações WPF: as **Views** (XAML) contêm apenas bindings e pequenos encaminhamentos de eventos de interface; as **ViewModels** orquestram os **Services** e expõem propriedades/comandos para binding; os **Services** concentram toda a regra de negócio (comunicação serial, rastreamento, seleção de torre, controle de acionamento, autenticação, permissões) sem depender da camada gráfica; e os **Models** são entidades de domínio simples. O mecanismo de binding MVVM do RadarTorres (**ViewModelBase**, **RelayCommand**) foi implementado manualmente, sem um framework externo como Prism ou CommunityToolkit.Mvvm — decisão documentada como restrição arquitetural em `Docs/Diagramas_e_requisitos/Decisoes_Arquiteturais.md`, tomada para manter o mecanismo de binding inteiramente explicável na defesa do trabalho. Qualquer menção à arquitetura do RadarTorres neste documento e nos diagramas associados refere-se exclusivamente a essa organização MVVM, nunca a MVC.

3 Diagramas UML utilizados no RadarTorres

Entre os diagramas UML disponíveis, três foram selecionados por representarem, de forma direta e com evidência no código-fonte, os aspectos mais relevantes do RadarTorres: o diagrama de casos de uso (quem interage com o sistema e para quê), o diagrama de classes (como o sistema está estruturado internamente) e o diagrama de implantação (onde e como os componentes físicos se comunicam). Diagramas de sequência, atividades ou máquina de estados não foram produzidos nesta etapa por não terem sido solicitados; nada impede que sejam adicionados posteriormente a partir da mesma base de requisitos, caso o TCC precise de um nível de detalhe comportamental adicional.

3.1 Diagrama de Casos de Uso

O diagrama de casos de uso é um diagrama comportamental que representa os objetivos que os atores de um sistema conseguem alcançar interagindo com ele, sem descrever os passos internos de como esses objetivos são alcançados. Seus elementos centrais são o **ator** (um papel externo ao sistema — uma pessoa, um perfil de usuário ou outro sistema — que inicia ou participa de uma interação), o **caso de uso** (um objetivo discreto e observável que o ator alcança) e a **fronteira do sistema** (o limite que separa o que é interno ao sistema modelado do que é externo a ele). Relacionamentos como `<<include>>` (um caso de uso incorpora obrigatoriamente outro como parte de seu fluxo) e `<<extend>>` (um caso de uso estende opcionalmente outro sob uma condição específica) permitem compor casos de uso complexos a partir de partes menores, sem repetir a mesma descrição em vários lugares.

Aplicado ao RadarTorres, o sistema foi modelado com uma única fronteira ("Sistema RadarTorres") e cinco atores. Um ator genérico **Usuário** foi introduzido, por generalização, como pai de **Administrador**, **Operador** e **Visualizador** — os três perfis reais definidos em `PerfilUsuario (Services/PermissionService.cs)` — porque a maioria dos casos de uso do sistema está disponível para qualquer perfil autenticado, e a generalização evita repetir a mesma associação três vezes onde ela não é semanticamente necessária. O **Arduino** aparece como ator externo apenas nos dois pontos em que realmente troca mensagens com o sistema pela porta serial (o envio de leituras de sensor e o tráfego observado no monitor serial da aba de configuração) — nunca como se fosse um usuário do sistema.

O diagrama mapeia 28 casos de uso, agrupados visualmente em seis blocos (Conta e Acesso; Monitoramento e Operação; Histórico e Dados; Administração; Preferências; Arduino e Comunicação) para manter a legibilidade. Cada caso de uso corresponde a um requisito funcional válido — nenhum botão isolado da interface foi elevado à condição de caso de uso. Os casos de uso que descrevem reação automática do sistema (rastrear alvos, selecionar torre, acompanhar o alvo, executar o acionamento demonstrativo, validar as regras de segurança) não têm um ator humano associado: são alcançados apenas por `<<include>>/<<extend>>` a partir de quem efetivamente inicia a cadeia — o Usuário observando o radar, ou o Arduino enviando uma leitura — reforçando visualmente que não existe, no sistema, um caso de uso de "acionar manualmente".

Diagrama fonte: `Diagramas/Casos_de_Uso_RadarTorres.puml`. A especificação textual completa de cada um dos 28 casos de uso (objetivo, atores, fluxos, requisito relacionado e status de implementação) está em `Docs/Diagramas_e_requisitos/Casos_de_Uso.md`.

4 Modos de operação

Um ponto de atenção específico do RadarTorres, relevante para a leitura de qualquer diagrama que envolva o comportamento de rastreamento e acionamento, é

o conjunto de modos de operação do sistema. A especificação de requisitos revisada (Docs/Diagramas_e_requisitos/Requisitos_Funcionais.md, RF08) define exatamente três estados:

- **Verde** — sistema ligado, porém sem operação funcional de rastreamento ou acionamento.
- **Amarelo** — o sistema realiza detecção e rastreamento de alvos, e as torres acompanham automaticamente o alvo selecionado; **não ocorre acionamento** em nenhuma hipótese.
- **Vermelho** — o sistema realiza detecção, rastreamento e acompanhamento automático pelas torres, e o acionamento demonstrativo ocorre **automaticamente**, quando permitido pelas regras de segurança (distância mínima, ausência de zona morta ativa e existência de torre selecionada). Não existe, em nenhum modo, um caminho de acionamento manual.

Todos os diagramas deste documento adotam exclusivamente essa nomenclatura de três estados. É importante registrar, por rigor acadêmico, que o código-fonte hoje ainda implementa esse comportamento por meio de um enumerador (`SystemMode`, em `Models/SystemState.cs`) com seis valores herdados de uma versão anterior do sistema (`Off`, `LocationOnly`, `LocationAutoTower`, `LocationAutoFire`, `Maintenance`, `Emergency`), e que o código ainda expõe um comando de acionamento manual (`MainViewModel.ManualFireCommand`) não gated por modo. Essa divergência entre a especificação revisada e a implementação atual está documentada em detalhe em Docs/Diagramas_e_requisitos/Limitacoes_Conhecidas.md (divergência D1) e não foi corrigida no código como parte desta tarefa, que é exclusivamente documental.

5 Diagrama de Classes

O diagrama de classes é um diagrama estrutural que descreve os tipos de objetos que compõem um sistema e os relacionamentos estáticos entre eles. Cada classe é representada por um retângulo dividido em três compartimentos: o nome da classe, seus **atributos** (as propriedades que caracterizam um objeto daquele tipo) e suas **operações** (os métodos que definem o comportamento disponível). Entre classes, a UML define diferentes tipos de relacionamento: **associação** (uma ligação estrutural simples entre duas classes, frequentemente anotada com **multiplicidade** — quantas instâncias de um lado se relacionam com quantas do outro), **herança**/generalização (uma classe especializa outra, herdando sua estrutura), e **dependência** (uma classe utiliza outra, tipicamente como parâmetro ou tipo de retorno de uma operação, sem mantê-la como atributo permanente).

O diagrama de classes do RadarTorres, disponível em Diagramas/Classes_RadarTorres.puml, é deliberadamente conceitual: não representa todas as classes do projeto (haveria dezenas, incluindo Views, Converters e Helpers,

sem ganho de compreensão para o leitor), mas concentra-se nas entidades de domínio e nos serviços centrais necessários para entender o funcionamento do sistema. Do lado das entidades de domínio estão **Target** (o alvo em rastreamento, com posição, quadrante e torre associada), **Tower** (torre demonstrativa configurável), **DeadZone** (área de exclusão), **SensorReading** (leitura bruta de sensor), **Usuario** (conta autenticável, com o atributo **Perfil** do tipo **PerfilUsuario**), e os quatro registros de auditoria — **ObjetoDetectado**, **AcaoRealizada**, **AlteracaoModo** e **ChamadoAjuda**. Do lado dos serviços estão as interfaces **ITargetTrackingService**, **ITowerSelectionService**, **IFireControlService**, **IDeadZoneService**, **ISerialCommunicationService**, **IAuthService** e **IPermissionService** — cada uma responsável por uma fatia específica da regra de negócio, seguindo a separação de responsabilidades já descrita na Seção 2.

Nota metodológica. Não existe, no código-fonte, uma classe chamada **SystemState**: esse é o nome do arquivo **Models/SystemState.cs**, que agrupa cinco enumeradores (**SystemMode**, **ConnectionState**, **TowerState**, **Quadrant**, **DataSource**), não uma entidade com atributos e identidade própria. O diagrama representa, em vez disso, o enumerador **SystemMode** diretamente, por ser o elemento real que expressa o modo de operação do sistema — evitando representar uma classe que não existe no projeto.

6 Diagrama de Implantação

O diagrama de implantação é um diagrama estrutural que descreve a topologia física de um sistema: quais nós de hardware existem, quais artefatos de software são executados em cada nó, e como os nós se comunicam entre si. É particularmente relevante para sistemas que combinam software e hardware, como é o caso do **RadarTorres**, em que a arquitetura lógica (Seção 2) não é suficiente para explicar onde o processamento efetivamente ocorre nem como a informação atravessa a fronteira entre o computador e o dispositivo físico.

O diagrama de implantação do **RadarTorres**, em **Diagramas/Implantacao_RadarTorres.puml**, representa dois nós: o **computador do usuário** (Windows 10/11, 64-bit), executando o artefato **RadarTorres.App** sobre o .NET 9 Desktop Runtime — embutido pelo instalador self-contained, sem dependência externa — e persistindo dados localmente em **%AppData%\RadarTorres\Data*.csv** e **%LocalAppData%\RadarTorres*.json**; e o **Arduino** (microcontrolador), executando o firmware responsável pela leitura dos sensores e pelo acionamento das torres demonstrativas. A comunicação entre os dois nós ocorre exclusivamente por **USB/serial**, no protocolo textual documentado em **Docs/COMUNICACAO_ARDUINO.md**: o computador envia comandos (configuração, acionamento) e o Arduino envia leituras de alvo. Sensores de detecção alimentam o Arduino, e as torres/indicadores demonstrativos (laser de baixa potência, LED ou simulação — nunca armamento real) são acionados pelo Arduino a partir do comando recebido do computador. Não há nenhum componente de servidor, web ou nuvem nessa topologia — deliberadamente, para não representar infraestrutura que o **RadarTorres** não possui.

7 Relação entre UML e MVVM

Os três diagramas apresentados não são independentes entre si: juntos, ajudam a visualizar como a organização MVVM do RadarTorres (Seção 2) se conecta à interação externa com o operador e com o Arduino. O diagrama de casos de uso mostra **o que** cada ator consegue fazer; o diagrama de classes mostra **como** essas capacidades são sustentadas pela cadeia `View ↔ ViewModel ↔ Services ↔ Models/Persistência` — as `Views` capturam a interação do ator, as `ViewModels` traduzem essa interação em chamadas aos `Services`, e os `Services` manipulam os `Models` e persistem o resultado; e o diagrama de implantação mostra **onde** fisicamente essa cadeia é executada, e como ela se estende, na outra ponta da comunicação serial, até o Arduino e os dispositivos físicos que ele controla. Essa tríade de diagramas — comportamento externo, estrutura interna e topologia física — cobre, de forma complementar, tanto o software quanto a integração com hardware que caracterizam o RadarTorres.